

树上倍增

给定任意节点 i ，可以快速查询，从 i 节点往上走的路径中位于第 s 层的编号
生成复杂度 $n \log (n)$,查询复杂度 $\log (n)$

```

#include<bits/stdc++.h>
using namespace std;
const int maxn=5e5+10;
int limit=32;
int power;
int log2(int x){
    int ans=0;
    while((1<<ans)<=(x>>1)){
        ans++;
    }
    return ans;;
}
struct tree{
    int n,t;
}tr[maxn];
int head[maxn],deep[maxn],st[maxn][limit],tot=1;
void add(int u,int v){
    tr[++tot].t=v;
    tr[tot].n=head[u];
    head[u]=tot;
}
void dfs(int i,int f){
    if(i==0){// 0节点为头节点
        deep[i]=1;
    }else deep[i]=deep[f]+1;
    st[i][0]=f;
    for(int p=1;(1<<p)<=deep[i];p++){
        st[i][p]=st[st[i][p-1]][p-1];
    }
    for(int ei=head[i];ei;ei=tr[ei].n){
        dfs(tr[ei].t,i);
    }
}
int getkthAncestor(int i,int k){
    if(deep[i]<=k) return -1;
    int s=deep[i]-k;
    for(int p=power;p>=0;p--){
        if(deep[st[i][p]]>=s){
            i=st[i][p];
        }
    }
    return i;
}

```

更快的查询方法,迭代版,防止爆栈

P5903

```
int find(int u,int k){
    int cnt = 0;
    while(k){
        if(k & 1) u = fa[u][cnt];
        k >>= 1;
        cnt++;
    }
    return u;
}
```

求LCA

```
int findLCA(int a, int b) {
    if (deep[a] < deep[b]) swap(a, b);
    // 将a上移到与b同深度
    for (int p = power; p >= 0; p--) {
        if (deep[st[a][p]] >= deep[b]) {
            a = st[a][p];
        }
    }
    if (a == b) return a;
    // 一起向上跳
    for (int p = power; p >= 0; p--) {
        if (st[a][p] != st[b][p]) {
            a = st[a][p];
            b = st[b][p];
        }
    }
    return st[a][0];
}
```

树的直径

```
// 树上最远的两个点的距离
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn=5e5+10;
int head[maxn*2],tot=1,n;
struct tree{
    int t,n,w;
}tr[maxn*2];
void add(int u,int v,int w){
    tr[++tot].t=v;
    tr[tot].n=head[u];
    tr[tot].w=w;
    head[u]=tot;
}
void build(){
    tot=1;
    memset(head,0,sizeof(head));
}
//1 两遍dfs
//不能有负边，能够得到具体两点
// int start,en,dis[maxn],last[maxn];
// void dfs(int u,int f,int w){
//     last[u]=f;
//     dis[u]=dis[f]+w;
//     for(int ei=head[u];ei;ei=tr[ei].n){
//         if(tr[ei].t!=f){
//             dfs(tr[ei].t,u,tr[ei].w);
//         }
//     }
// }
// }
// int main()
// {
//     cin>>n;
//     build();
//     for(int i=1;i<n;i++){
//         int u,v,w;
//         cin>>u>>v>>w;
//         add(u,v,w);
//         add(v,u,w);
//     }
```

```

//    dfs(1,0,0);
//    start=1;
//    for(int i=1;i<=n;i++){
//        if(dis[i]>dis[start]){
//            start=i;
//        }
//    }
//    dfs(start,0,0);
//    en=1;
//    for(int i=1;i<=n;i++){
//        if(dis[i]>dis[en]){
//            en=i;
//        }
//    }
//    //cout<<start<<' '<<en<<endl;
//    cout<<dis[en]<<endl;
// }
// 2 树形dp
//不能得到具体的两点
// int dis[maxn],ans[maxn];
// void dp(int u,int f){
//     for(int ei=head[u];ei;ei=tr[ei].n){
//         int v=tr[ei].t,w=tr[ei].w;;
//         if(f!=v){
//             dp(v,u);
//             ans[u]=max(ans[u],dis[u]+dis[v]+w);
//             dis[u]=max(dis[u],dis[v]+w);
//         }
//     }
// }
// }
// int main(){
//     cin>>n;
//     build();
//     for(int i=1;i<n;i++){
//         int u,v,w;
//         cin>>u>>v>>w;
//         add(u,v,w);
//         add(v,u,w);
//     }
//     int Ans=0;
//     dp(1,0);
//     for(int i=1;i<=n;i++){

```

```

//      Ans=max(Ans,ans[i]);
//      }
//      cout<<Ans<<endl;
//  }

// 3 如和求所有直径的公共部分，使用方法一
ll start,en,dis[maxn],last[maxn];
bool path[maxn];
ll coedge;
void dfs(int u,int f,int w){
    last[u]=f;
    dis[u]=dis[f]+w;
    for(int ei=head[u];ei;ei=tr[ei].n){
        if(tr[ei].t!=f){
            dfs(tr[ei].t,u,tr[ei].w);
        }
    }
}
ll maxdis(int u,int f,ll c){
    ll ans=c;
    for(int ei=head[u];ei;ei=tr[ei].n){
        if(tr[ei].t!=f&&!path[tr[ei].t]){
            ans=max(ans,maxdis(tr[ei].t,u,c+tr[ei].w));
        }
    }
    return ans;
}
int main()
{
    cin>>n;
    build();
    for(int i=1;i<n;i++){
        int u,v,w;
        cin>>u>>v>>w;
        add(u,v,w);
        add(v,u,w);
    }
    dfs(1,0,0);
    start=1;
    for(int i=1;i<=n;i++){
        if(dis[i]>dis[start]){
            start=i;
        }
    }
}

```

```

}
dfs(start,0,0);
en=1;
for(int i=1;i<=n;i++){
    if(dis[i]>dis[en]){
        en=i;
    }
}
for(int i=en;i=i=last[i]){
    path[i]=1;
}
int l=start,r=en;
ll maxd;
for(int i=last[en];i!=start;i=last[i]){
    maxd=maxdis(i,0,0);
    if(maxd==dis[en]-dis[i]){
        r=i;
    }
    if(maxd==dis[i]&& l==start){
        l=i;
    }
}
for(int i=r;i!=l;i=last[i]) coedge++;
cout<<dis[en]<<endl<<coedge;
}

```